



PointsBoard v11.2.1

Beginner Setup Guide

GitHub source control + Cloudflare Workers deployment

Release	11.2.1 - deployment-audited
Guide date	July 24, 2026
Recommended host	Cloudflare Workers Builds
Skill level	Beginner - follow the numbered steps

Important: never place API keys in GitHub or browser code.

1. What you are setting up

PointsBoard is a full-stack web application. GitHub stores the source code and runs verification checks. Cloudflare builds the React interface, serves the static files, and runs protected API routes that call seats.aero and SerpApi without exposing the provider keys to the browser.

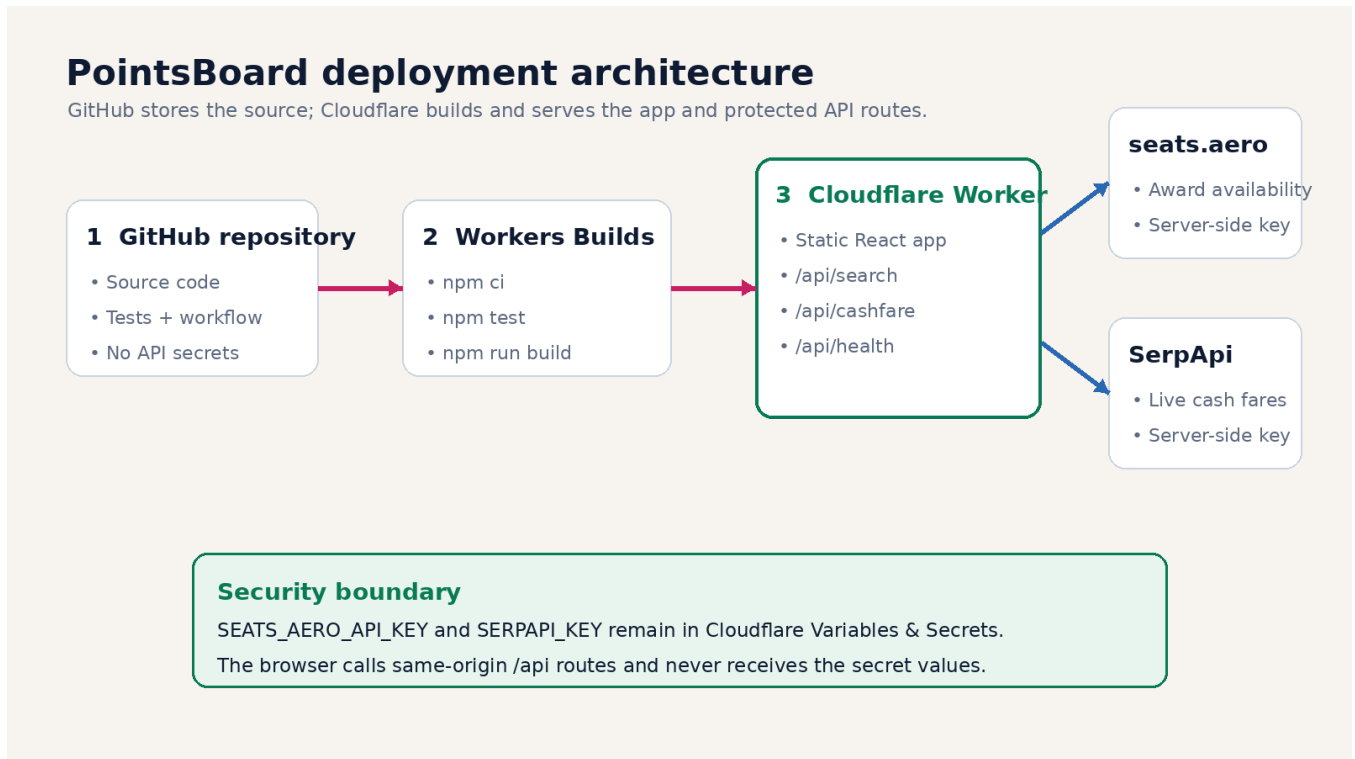


Figure 1. Recommended production architecture.

What the finished deployment should do

- Open from a public Cloudflare URL.
- Show Demo data only when the Demo switch is enabled.
- Use live seats.aero award data when Demo is disabled.
- Use live SerpApi cash fares when configured; otherwise show cash fare as unavailable.
- Keep non-USD taxes in their original currency until you enter a manual USD FX rate.
- Display a safe health page at /api/health.

2. What you need before starting

Item	Why it is needed
GitHub account	Stores the source and runs the included verification workflow.
Cloudflare account	Builds and hosts the Worker and the static application.
seats.aero Partner API key	Required for live award availability.
SerpApi key	Required for live cash fares and cash-based CPP.
The extracted PointsBoard package	Use the folder that contains package.json and wrangler.jsonc.

3. Confirm the package structure

After extracting the ZIP, open the project folder. The following items must be together at the top level. Do not upload the outer ZIP file as a single file.

Look for	Meaning
package.json	Required at repository root
package-lock.json	Required at repository root
wrangler.jsonc	Required at repository root
src/	Required project folder/file
worker/	Required project folder/file
functions/	Required project folder/file
public/	Required project folder/file
.github/	Required project folder/file
START_HERE.md	Required project folder/file

Do not upload: node_modules, dist, .dev.vars, .wrangler, or .wrangler-dry-run.

4. Put the project on GitHub

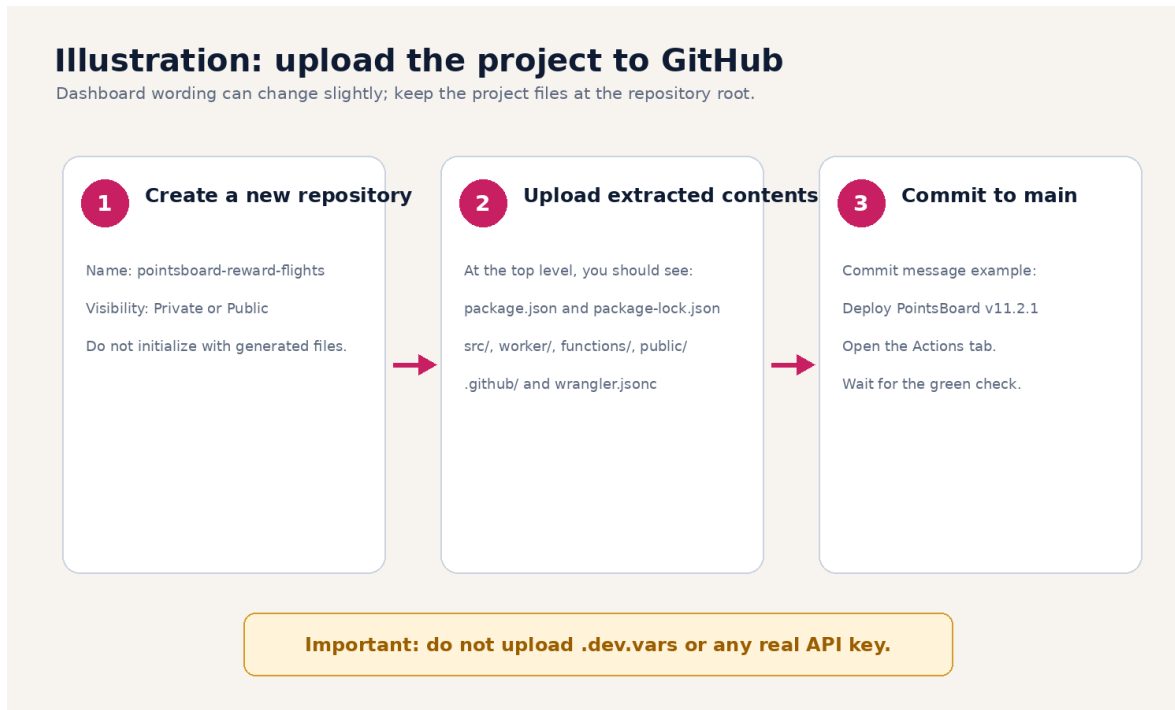


Figure 2. GitHub upload sequence (illustration; dashboard labels can vary).

Method A - GitHub website

- 1. Create the repository** - In GitHub, choose New repository. Use the name pointsboard-reward-flights. Private or Public both work.
- 2. Avoid extra starter files** - When creating an empty repository for an existing project, do not add a second README, .gitignore, or license if GitHub gives you that option.
- 3. Upload project contents** - Choose Add file > Upload files. Drag the extracted contents, not the ZIP itself. Confirm package.json appears at the repository root.
- 4. Commit to main** - Use a message such as Deploy PointsBoard v11.2.1 and commit the files.
- 5. Open Actions** - The included Verify PointsBoard workflow should start. A green check means the clean install, tests, Vite build, and Wrangler dry run all passed in GitHub.

Method B - command line

Use this method only if you are comfortable with Terminal or PowerShell. Replace the example repository address with your own.

```

git init
git add .
git commit -m "Deploy PointsBoard v11.2.1"
git branch -M main
git remote add origin https://github.com/YOUR-ACCOUNT/pointsboard-reward-flights.git
git push -u origin main
  
```

5. Connect GitHub to Cloudflare Workers Builds

Illustration: connect GitHub to Cloudflare Workers Builds

Use Workers Builds, not a static-only Pages deployment, because this app has protected API routes.

Repository	your-account / pointsboard-reward-flights
Production branch	main
Root directory	Leave blank when package.json is at repository root
Build command	npm run build
Deploy command	npx wrangler deploy
Preview deploy command	npx wrangler versions upload

Worker name in wrangler.jsonc: pointsboard-reward-flights

Figure 3. Cloudflare Workers Builds settings (illustration; labels can vary).

1. **Open Cloudflare:** Sign in, open Workers & Pages, then choose Create application or Import repository.
2. **Connect GitHub:** Authorize the Cloudflare GitHub integration. Grant access to the PointsBoard repository.
3. **Select the repository:** Choose pointsboard-reward-flights and production branch main.
4. **Set the root directory:** Leave it blank when package.json is at the repository root. Enter the subfolder only if you intentionally uploaded the project inside a subfolder.
5. **Set the build command:** Use npm run build.
6. **Set the deploy command:** Use npx wrangler deploy.
7. **Set the preview command:** Use npx wrangler versions upload for non-production branches if the dashboard requests it.
8. **Confirm the Worker name:** Use pointsboard-reward-flights. It matches wrangler.jsonc.
9. **Save:** Do not start Live use until the secrets in the next section are added.

Why Workers Builds instead of GitHub Pages alone?

GitHub Pages is static hosting. PointsBoard also needs server-side API routes so the seats.aero and SerpApi keys remain private. Cloudflare Workers serves the app and the protected API routes in one deployment.

6. Add the encrypted API secrets

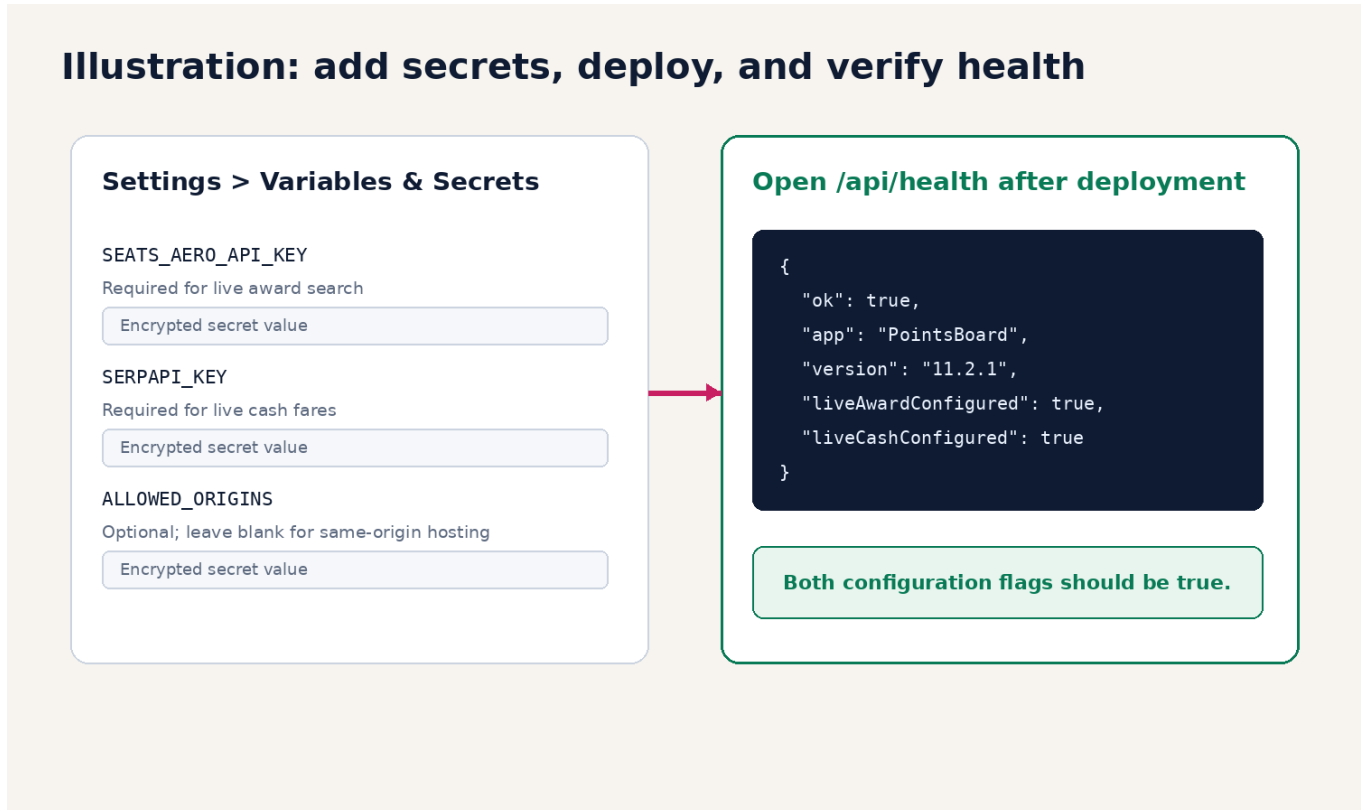


Figure 4. Add secrets and verify the health endpoint (illustration).

In the Cloudflare Worker, open Settings > Variables & Secrets. Add these as encrypted secrets, not ordinary text committed to GitHub:

Name	Required?	Purpose
SEATS_AERO_API_KEY	Yes for Live awards	Calls the seats.aero Partner API.
SERPAPI_KEY	Yes for Live cash fares	Calls SerpApi Google Flights.
ALLOWED_ORIGINS	Usually no	Optional comma-separated allowlist when the frontend is hosted on another origin.

Recommended: leave ALLOWED_ORIGINS blank when the app and Worker use the same Cloudflare domain.

7. Deploy and verify

- ❑ **Cloudflare build** - The build log should complete npm install/ci, npm run build, and npx wrangler deploy without a red error.
- ❑ **Open the app** - Use the Cloudflare workers.dev address. The PointsBoard interface should load.
- ❑ **Check health** - Add /api/health to the deployed address.
- ❑ **Confirm configuration** - liveAwardConfigured should be true. liveCashConfigured should also be true when SerpApi is configured.
- ❑ **Test Demo mode** - Turn Demo on and load a pre-built route.
- ❑ **Test strict Live mode** - Turn Demo off. Demo rows should disappear. Run a small live search.
- ❑ **Test unavailable cash behavior** - If SerpApi is missing or returns no fare, the app should say unavailable and must not create an estimated cash fare.
- ❑ **Test foreign fees** - Use a Demo itinerary with GBP, EUR, JPY, or another non-USD fee. CPP should remain unavailable until an FX rate is entered.

Expected health response

```
{
  "ok": true,
  "app": "PointsBoard",
  "version": "11.2.1",
  "liveAwardConfigured": true,
  "liveCashConfigured": true
}
```

8. How automatic updates work

After the GitHub repository is connected, a push to the production branch triggers Cloudflare Workers Builds. GitHub Actions separately runs the included verification workflow. Use this simple update routine:

10. Make or upload the change in a new branch when possible.
11. Wait for the GitHub Actions green check.
12. Review the Cloudflare preview deployment for a non-production branch.
13. Merge or push to main only after review.
14. Confirm the production build and /api/health after deployment.

9. What the application should look like

POINTSBOARD v11.1 DEMO · PRE-BUILT DATA Demo data

Strict mode separation: Demo mode uses only clearly tagged local scenarios. Turning Demo off removes every synthetic result and shows no fallback data.

RECOMMENDATION SETTINGS

Route: LAX → LHR · Business

Priority preset: Balanced

Maximum stops: Up to 1

Preferred layover hours: 1.25 4

Departure window: 6 22

Start later than end crosses midnight.

Recalculate

CPP source
1 JSON
Canonical library

Cash provenance
2 exact
Others labeled

Eligible options
4
After preferences

Mode integrity
Strict
No demo fallback in Live

RECOMMENDED REDEMPTIONS CPP library · 2026-07-01

LAX→LHR · demo date · business

BEST OVERALL 99/100

United MileagePlus

LAX→LHR · 17:30–12:00+1
Nonstop · 10.5h

Redeem	Economic cost
80,000 pts + \$6	\$1,046
Realized CPP	Confidence
5.18%	High

Cash basis: Exact itinerary fare

- Nonstop itinerary
- Departure fits selected window
- Cash provenance is explicitly labeled

LOWEST ECONOMIC COST 99/100

United MileagePlus

LAX→LHR · 17:30–12:00+1
Nonstop · 10.5h

Redeem	Economic cost
80,000 pts + \$6	\$1,046
Realized CPP	Confidence
5.18%	High

Cash basis: Exact itinerary fare

- Nonstop itinerary
- Departure fits selected window
- Cash provenance is explicitly labeled

BEST REALIZED CPP 66/100

Air Canada Aeroplan

LAX→LHR · 11:20–09:15+1
1 stop · 12.9h

Redeem	Economic cost
70,000 pts + \$86	\$1,136
Realized CPP	Confidence
5.81%	Medium

Cash basis: Route/cabin median benchmark

- Layover within preferred range
- Departure fits selected window
- Cash provenance is explicitly labeled

Figure 5. PointsBoard recommendation interface. The exact layout can vary by browser width.

The important indicators are the Demo/Live mode badge, strict mode separation message, canonical CPP library status, cash provenance, eligible option count, recommendation cards, and clearly separated cash fare versus economic cost.

10. Independent audit summary

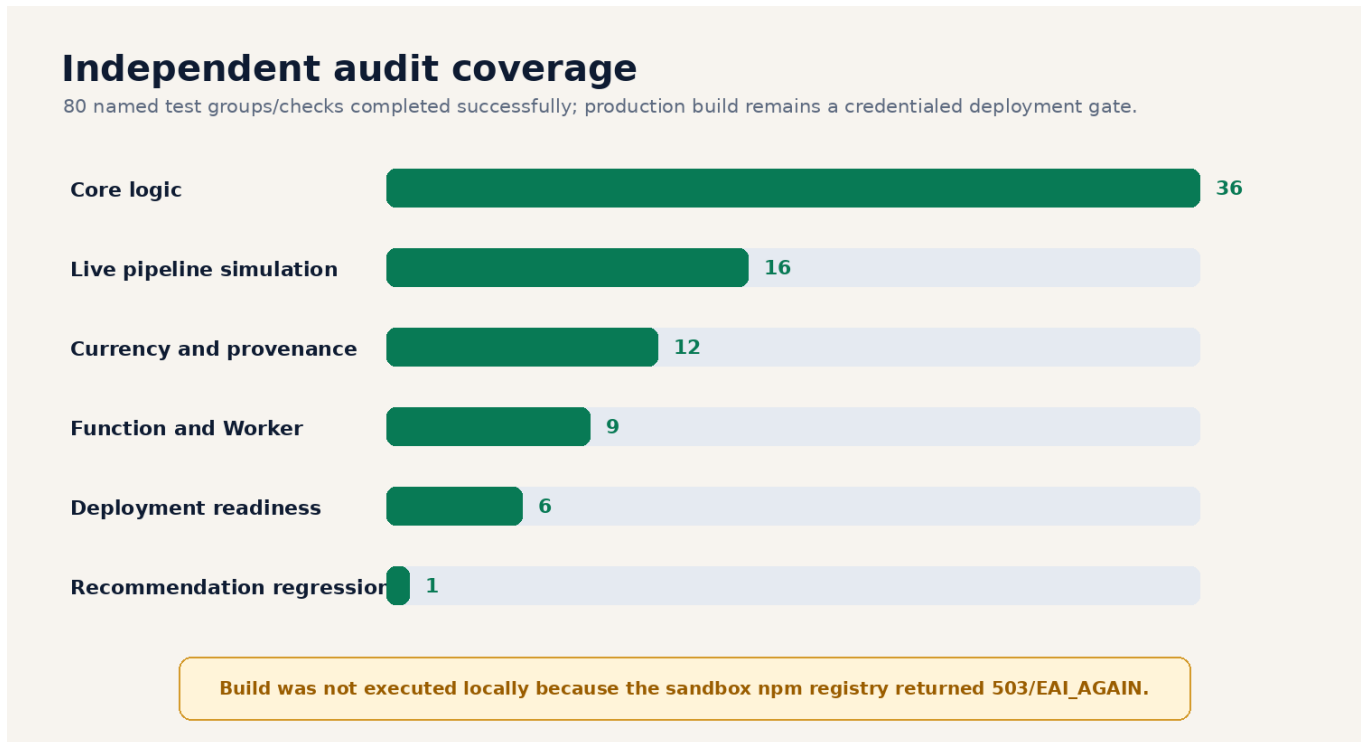


Figure 6. Audit coverage and remaining external deployment gate.

All 80 named source-level test groups/checks passed. A material Live-mode estimate inconsistency was found and corrected. The package is approved for staged deployment after the first green GitHub Actions run and Cloudflare Workers Build.

What was not possible inside the audit sandbox

The audit environment could not download Vite/Wrangler dependencies because its internal npm mirror returned HTTP 503 and direct access returned DNS EAI_AGAIN. Therefore, the audit did not perform a credentialed Cloudflare production deployment. The included GitHub Actions workflow and Cloudflare Workers Build must complete the clean install and build in the user-connected environment.

11. Troubleshooting

Symptom	What to do
GitHub Actions fails at npm ci	Confirm package-lock.json is at the root and was uploaded. Retry once if npm has a temporary outage.
Cloudflare says it cannot find package.json	Root directory is wrong. Leave it blank if package.json is at repository root.
Worker name conflict or mismatch	Use pointsboard-reward-flights or change the

Symptom	What to do
	name consistently in Cloudflare and wrangler.jsonc.
/api/health shows liveAwardConfigured false	Add or correct SEATS_AERO_API_KEY in Cloudflare Variables & Secrets, then redeploy.
/api/health shows liveCashConfigured false	Add or correct SERPAPI_KEY, then redeploy. Awards can still work without live cash fares.
Award search returns 500	Check the seats.aero key and Partner API access.
Cash fare is unavailable	Check SERPAPI_KEY, quota, and provider response. The app intentionally does not invent a fare.
CPP is blank for foreign fees	Enter a manual USD FX rate for the original fee currency.
Browser shows an old version	Wait for deployment completion, refresh, or use a private browsing window.
Cross-origin API request is blocked	For same-domain hosting, leave ALLOWED_ORIGINS blank. For a separate frontend, add only the exact trusted origin.

12. Security checklist

- ☐ No API keys are stored in GitHub.
- ☐ .dev.vars remains local and ignored.
- ☐ Cloudflare secrets are encrypted.
- ☐ ALLOWED_ORIGINS is blank unless deliberately needed.
- ☐ The health endpoint shows only true/false configuration state.
- ☐ GitHub Actions and Cloudflare builds are green before production use.
- ☐ Keys are rotated immediately if accidentally exposed.

13. Official reference sources

Cloudflare Workers Builds: <https://developers.cloudflare.com/workers/ci-cd/builds/>

Cloudflare Workers Builds configuration:

<https://developers.cloudflare.com/workers/ci-cd/builds/configuration/>

Cloudflare GitHub integration: <https://developers.cloudflare.com/workers/ci-cd/builds/git-integration/github-integration/>

Cloudflare Static Assets: <https://developers.cloudflare.com/workers/static-assets/>

Cloudflare Single Page Application routing: <https://developers.cloudflare.com/workers/static-assets/routing/single-page-application/>

Cloudflare Wrangler commands: <https://developers.cloudflare.com/workers/wrangler/commands/>

GitHub - adding locally hosted code: <https://docs.github.com/articles/adding-an-existing-project-to-github-using-the-command-line>

Final gate: deploy first to staging, verify GitHub Actions, Cloudflare Build, /api/health, Demo/Live separation, and one live route before treating the app as production-ready.